

CSED441 Assignment 1 Report

20220019 안재영

1. Assignment overview

본 과제는 다수(3개)의 일부가 겹치는 이미지들을 자연스럽게 결합하여 파노라마 이미지를 생성하는 것을 목표로 하고 있다. 구현은 크게 4가지 단계로 구성되며, 각각에 대한 설명은 다음과 같다.

1. SIFT: 이미지 크기 및 변환에 강건한 Keypoint를 검출하고, 서로 다른 이미지에서 Keypoint를 매칭하기 위한 Descriptor를 추출한다.

2. Matching: 이웃한 두 이미지의 Descriptor를 사용하여 대응되는 Keypoint 쌍을 찾고, 상호 검증 단계를 거친다.

3. RANSAC + Homography: 매칭된 Keypoint를 사용해 이미지를 겹치기 위한 자연스러운 호모그래피 행렬을 유도한다. 이 때 정확도를 높이기 위해 RANSAC 알고리즘을 활용한다.

4. Warping, Blending & Panorama: 유도한 호모그래피 행렬을 사용한 역방향 매핑과 선형 보간을 통해 이미지들을 파노라마 캔버스에 배치하여 파노라마 이미지를 생성한다.

2. Implementation and Algorithm

Stage 1. SIFT

`gaussian.py`

`gaussian_kernel_1d` 함수는 1차원 가우시안 커널을 생성하는 함수이다. 표준편차와 반지름을 입력 받고, 그 값들을 사용해 가우시안 분포를 따르는 1차원 배열을 생성한다. 이후 그 합을 계산하고 각 커널 값을 합으로 나누어 L1 정규화를 수행한다.

`image_gradients` 함수는 가우시안 미분(DoG)를 기반으로 이미지의 기울기를 계산하는 함수이다. DoG는 먼저 가우시안 필터로 이미지를 스무딩하여 노이즈의 영향을 줄인 후, 미분 커널을 적용해 기울기를 구한다. 이 함수는 가로 기울기 I_x 와 세로 기울기 I_y 를 분리 가능하도록 별도로 계산해 반환한다.

가우시안 필터는 앞서 구현한 `gaussian_kernel_1d` 함수를 호출해 사용하며, 미분 커널은

중심 차분 커널인 $[-0.5, 0.0, 0.5]$ 를 사용한다. `seperable_conv2d` 함수를 사용해 가우시안 커널을 적용한 후, smoothing된 이미지의 가로 방향($axis = 1$)에 미분 커널을 적용해 I_x 를, 세로 방향($axis = 0$)에 미분 커널을 적용해 I_y 를 도출한다.

dog.py

이 파일 내의 함수는 이미지 내에서 Scale-Invariant한 극값을 찾기 위한 DoG 피라미드를 생성하여 SIFT 알고리즘의 핵심인 keypoint 후보를 찾는 기능을 수행한다.

`build_gaussian_pyramid` 함수는 그 첫 단계로 가우시안 피라미드를 구축한다. 옥타브와 스케일로 구성된 계층적 구조를 생성하며, 인접한 스케일 레벨 간의 시그마 증가 비율은 $k = 2^{(1/scales)}$ 에 의해 결정된다. 피라미드는 옥타브와 스케일로 구성된 계층적 구조로 생성되며, 바깥 루프인 옥타브 루프는 이미지의 해상도를 절반으로 줄이면서 다음 옥타브를 시작한다. 구체적으로, 다음 옥타브의 베이스 이미지는 이전 옥타브의 중간 레벨 이미지를 2:1 비율로 다운샘플링하여 얻게 된다. 안쪽 루프인 스케일 루프는 keypoint의 검출에 필요한 $scales+2$ 개의 DoG 레벨을 생성하기 위해 각 옥타브 내에서 $scales+3$ 개의 스케일 레벨을 생성한다. 각 스케일 레벨은 이전 레벨의 이미지에 $\sqrt{\sigma^2 - \text{prev_sigma}^2}$ 으로 계산되는 σ_{blur} 로 증분 블러를 적용해 효율성을 높인다.

keypoint_filter.py

이 파일은 SIFT 알고리즘에서 keypoint 후보를 걸러내는 두 번째 필터링 단계인 Edge Response Rejection을 구현한다. 이는 DoG 피라미드에서 찾은 극값 중 Edge에 위치한 불안정한 점들을 제거하는 역할을 한다.

`hessian_edge_reject` 함수는 2차원 헤시안 행렬을 사용해 keypoint 후보 중 엣지 상에 있는 점을 판별하고 제거한다. DoG 이미지 D 의 픽셀값을 이용해 2차 미분인 D_{xx} , D_{yy} , D_{xy} 를 유한 차분 방식으로 근사하고, 이 값들은 $[[D_{xx}, D_{xy}], [D_{xy}, D_{yy}]]$ 의 형태로 헤시안 행렬을 구성한다. edge score는 헤시안 행렬의 트레이스 $\text{Tr}(H)$ 와 행렬식 $\text{Det}(H)$ 에 대해 $\text{Tr}(H)^2 / \text{Det}(H)$ 의 식을 따르며, $\alpha/\beta=r$ 인 경우의 임계값 $\frac{(r+1)^2}{r}$ 과 비교하여 값이 클 경우 제거할 엣지로 간주되어 false를 반환해야 한다. 추가로 행렬식이 0일 경우 edge score의 계산이 불가능하므로, 두 가지의 경우를 and 연산하여 반환한다.

orientation.py

이 파일은 SIFT 알고리즘의 세 번째 단계인 Orientation Assignment 기능을 구현한다. 앞

서 검출 및 검증이 완료된 Keypoint에 Dominant Orientation을 부여해 회전 불변성을 가지도록 한다.

orientation_assignment 함수는 하나의 keypoint 후보를 입력받아 그 주변 영역의 기울기 방향을 분석하고, 가장 많이 나타나는 방향을 해당 keypoint의 주 방향으로 할당하는 함수이다. 처음부터 구현되어 있는 부분은 keypoint의 옥타브(o)와 스케일(s)에 해당하는 가우시안 피라미드 이미지를 G에 저장하고 image_gradients 함수를 호출하여 그 가로, 세로의 기울기 성분 I_x , I_y 를 계산한다. 그 후 두 기울기 성분을 사용해 각 픽셀의 기울기 크기(magnitude)와 기울기 방향(angle)을 계산한다. m과 a는 keypoint 중심의 (y, x) 주변의 17x17 픽셀에서 기울기 크기 및 기울기 방향을 추출한 후, 크기에는 시그마=1.5인 1차원 가우시안 커널을 사용해 가우시안 smoothing을 적용한 것이다. 이를 통해 keypoint 중심에 가까울수록 더 큰 가중치를 부여하여 방향 결정에 노이즈가 미치는 영향을 줄일 수 있다.

이 함수에서 새롭게 구현한 부분은 가중치를 고려해 계산된 지역 기울기 크기와 방향 데이터를 사용해 방향 히스토그램을 만들고 가장 큰 피크를 찾아 키포인트 방향으로 할당하는 부분이다. hist에 num_bins를 인자로 하는 0으로 초기화된 배열을 할당하고, 360도를 num_bins로 나눈 값을 bin_width로 설정한다. 앞서 계산한 방향은 0~2pi 범위로 정규화한다.

이후 히스토그램을 채우는 과정에서는 추출된 윈도우 내의 모든 픽셀을 순회하면서 픽셀 방향을 bin_width로 나눈 정수 부분을 해당 방향이 속하는 인덱스로 지정한다. 그리고 해당 인덱스의 히스토그램에 기울기 크기를 합산하여 인덱스의 투표 강도를 반복적으로 갱신한다.

히스토그램을 완성한 후 maxval에 가장 높은 투표 강도의 값을 저장하고, 그 값의 80% 이상인 모든 히스토그램 구간을 주 방향으로 간주한다. 조건을 만족하는 모든 방향을 주 방향으로 삼으며, orientations에는 각 주 방향에 대한 theta가 계산되어 저장된다.

descriptor.py

이 파일은 SIFT 알고리즘의 마지막 단계인 Descriptor 생성을 수행한다. Descriptor는 검출된 keypoint들에 대한 정보를 벡터로 요약하여 다른 이미지의 keypoint와 매칭할 수 있는 기반을 만든다.

sift_descriptor 함수는 keypoint를 전달받아 keypoint가 속한 가우시안 피라미드 이미지에서 크기 H, W와 기울기 크기와 방향을 계산한다. 이때 방향은 이전 단계에서 저장해둔 주 방향 theta를 빼는 방식으로 좌표계를 회전하여, 기준 방향을 0으로 만들어 회전 불

변성을 확보하고 있다.

다음으로는 $\text{win} = \text{cell} * \text{scale}$ 로 계산한 윈도우 크기를 사용해 keypoint 중심으로 가로 세로 win만큼을 윈도우로 결정하고 해상 영역에서 기울기 크기 m 과 방향 a 를 추출한다. 이 윈도우는 cell을 사용해 계산된 격자 간격 $hstep, wstep$ 을 통해 cell x cell의 격자로 나누어져 bins개의 방향 빈까지 포함된 descriptor에 저장될 예정이다.

이어지는 구현할 부분이 셀을 순회하며 히스토그램을 구축하는 과정으로, cell x cell을 순회하는 이중 반복문을 수행한다. 반복문 안에서는 윈도우에서 현재 위치의 셀 속 데이터를 m_cell 과 a_cell 에 저장하고, 360도를 방향 빈 개수로 나눈 방향 히스토그램에 a_cell 을 인덱스로, m_cell 을 가중치로 사용하여 저장한다.

마지막 구현 파트는 반복문을 통해 구성된 descriptor를 후처리하는 과정이다. flatten 메서드를 적용해 cell x cell x bins 차원의 배열을 1차원 벡터로 만들고, norm을 계산한다. 이 norm 값을 사용해 원소들을 나눔으로써 벡터를 정규화하는데, 이때 SIFT 표준에 따라 노이즈와 비선형 조명의 효과를 최소화하기 위해 각 성분을 최대 0.2로 클리핑하는 기능을 추가로 구현하였다.

Stage 2. Matching

matcher.py

이 파일은 SIFT에 의해 생성된 descriptor를 사용해 두 이미지의 keypoint를 매칭하는 기능을 수행한다. 구체적으로 `match_l2_crosscheck` 함수가 L2-NN 매칭을 기반으로 keypoint를 매칭하며, Mutual cross-check로 결과를 검증한다. 이 함수는 두 descriptor 리스트를 `descA, descB`를 각각 배열에 저장하고, $|a-b|^2 = |a|^2 + |b|^2 - 2a \cdot b$ 에 따라 각 벡터들 사이의 pairwise squared-distance matrix를 효율적으로 계산할 수 있다. `argmin` 메서드를 통해 계산된 행렬의 두 방향에서 최근접 이웃의 인덱스를 찾고, 열 인덱스 `nnA`는 A->B 방향의 최근접 이웃, 행 인덱스 `nnB`는 B->A 방향의 최근접 이웃을 의미한다. 이때 올바른 매칭이 되기 위해서는 한 방향의 최근접 이웃으로 발견된 쌍이 반대쪽 방향에서도 최근접 이웃을 만족해야 한다. 이를 위해 `nnA`의 원소들을 순회하며 `nnB`에서도 동일한 쌍이 발견되는지 검사하고, 이 상호 검사를 통과한 쌍에 대해서 인덱스와 거리를 `matches`에 추가한다.

Stage 3. RANSAC + Homography

dlt.py

이 파일은 매칭된 keypoint 쌍을 이용해 두 이미지 사이의 호모그래피 행렬을 추정하는 기능을 수행한다. 추정을 위해서는 4개 이상의 쌍이 필요하기 때문에, 점들의 개수가 4 이상인지 확인한 후 알고리즘이 이어진다.

이미 작성되어 있는 `normalize_points` 함수는 입력 좌표 집합을 정규화하며, 이어진 `dlt_homography` 함수가 Direct Linear Transform 방법을 통해 호모그래피 행렬을 추정한다. 호모그래피 변환은 $x_2 \sim Hx_1$ 의 관계를 가지므로, 이를 선형화하면 한 keypoint 쌍 $x_1 = (x,y)^T$ 와 $x_2 = (u,v)^T$ 에 대해서 아래 두 방정식을 유도할 수 있다.

$$\begin{aligned}(h_{21}x + h_{22}y + h_{23}) - v(h_{31}x + h_{32}y + h_{33}) &= 0 \\ u(h_{31}x + h_{32}y + h_{33}) - (h_{11}x + h_{12}y + h_{13}) &= 0\end{aligned}$$

이 방정식들을 호모그래피 행렬 H 의 각 원소 $h_{11}, h_{12}, \dots$ 를 순서대로 나열한 9×1 행렬 h 에 대해 $Ah=0$ 의 형태로 표현할 수 있고, 따라서 행렬 A 의 두 행을 아래와 같이 구성한다.

$$[0, 0, 0, -x, -y, -1, v^*x, v^*y, v]$$

$$[x, y, 1, 0, 0, 0, -u^*x, -u^*y, -u]$$

다음으로는 특이값 분해(SVD)를 통해 방정식의 해를 구한다. `np.linalg`의 `svd` 메서드를 A 에 적용해 $S = USV^T$ 를 만족하는 행렬들을 구할 수 있고, 구하고자 하는 해 h 는 A 의 Null Space를 이루는 벡터이므로 가장 작은 특이값에 해당하는 v^T 의 행으로 주어진다. h 를 구한 후에는 3×3 행렬로 재구성 및 이전에 `normalize_points` 함수의 반환으로 저장한 정규화 행렬을 사용한 역정규화 과정을 거친다. 호모그래피 행렬은 사영 변환으로 전체 행렬에 상수를 곱해도 동일한 결과를 보장하므로, $H[2,2] \sim 1$ 이 되도록 스케일을 고정한 후 결과를 반환한다.

ransac.py

이 파일은 매칭된 keypoint 쌍들 중 잘못된 outlier들의 영향을 제거하여 정확한 호모그래피 행렬을 계산할 수 있도록 하는 RANSAC 알고리즘을 수행한다. RANSAC은 전체 데이터에서 무작위의 작은 샘플을 추출해 모델을 피팅하고, 해당 모델이 잘 적용되는 inlier의 비율을 산출하는 방식을 여러 번 반복해 최적의 모델을 선택하는 알고리즘이다. 반복은 `iters`의 횟수만큼 수행되며, 각 반복 내에서 호모그래피 행렬을 추정하는 데에 필요한 최소 조건인 4개의 점을 무작위로 선택한다. `dlt_homography` 함수를 호출해 행렬 H 를

계산하고, reprojection_errors 함수로 구한 에러가 정해진 thresh보다 작은 점들을 사용해 score를 계산한다. 이전까지의 반복에서 기록된 best_score보다 더 좋은 점수가 나올 경우, 이번에 계산된 H를 최적의 모델로 갱신한다.

정해진 횟수 iters만큼의 반복이 종료되면, 가장 좋은 점수를 기록한 행렬의 inlier들을 사용한 DLT로 최종 호모그래피 행렬을 계산해 정확도를 조금 더 높인다.

Stage 4. Warping, Blending & Panorama

warp.py

이 파일은 이미지 정렬 및 합성의 마지막 단계를 수행한다. 추정된 호모그래피 행렬을 적용하여 원본 이미지들을 타겟 캔버스의 적합한 위치에 배치하여 합친다.

warp_inverse_map 함수가 호모그래피 행렬을 사용한 역방향 매핑과 화질 손상을 최소화하기 위한 선형 보간을 함께 수행한다. 역방향 매핑은 원본 픽셀에서 대상 픽셀로 정방향 워핑을 수행할 때 발생할 수 있는 중복이나 구멍의 문제를 해결하기 위한 방법으로, 대상 픽셀을 순회하며 호모그래피의 역행렬을 사용해 원본 이미지에서 대응되는 픽셀을 찾는다. 또한 계산된 좌표들은 주로 정수가 아니기 때문에, 주변 픽셀들의 값을 조합하여 정수 좌표의 색상을 결정하는 보간 방식이 사용된다.

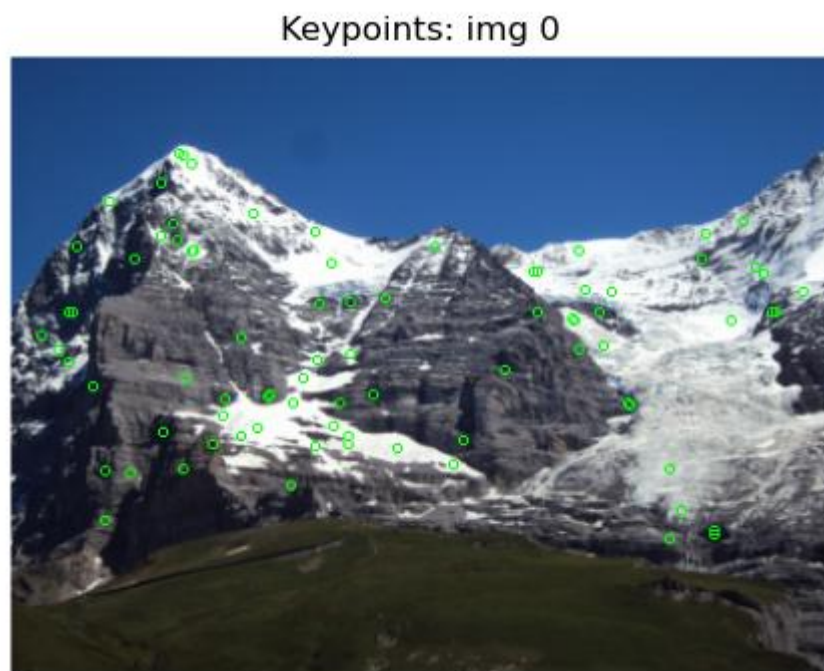
ys, xs에 출력 캔버스의 모든 정수 좌표에 해당하는 격자를 Hh, Hw의 크기에 맞게 생성하고, 각 격자 좌표들을 3 x N 크기의 동차 좌표 배열로 만들어 대상 좌표계를 준비한다. 호모그래피 행렬의 역행렬을 Hinv에 저장하고 이를 사용해 대상 좌표계의 점들에 대응되는 원본 이미지의 좌표를 src_coords에 연산한 후 마지막 성분(w)으로 나누어 (x/w, y/w) 꼴의 비동차 좌표로 바꿔준다. 이 좌표는 x_src와 y_src에 각각 원본 이미지의 x, y 대응 좌표로 저장된다. 이어지는 과정에서는 대상 좌표의 각 점을 순회하며 앞서 구한 원본 이미지의 대응 좌표를 사용해 각 픽셀의 색을 결정한다. 선형 보간을 사용하여 가장 가까운 정수 좌표 4개의 픽셀 값을 선형 결합으로 가중 평균하여 계산하며, 구체적인 수식은 아래와 같다. v 값들은 np.floor 메서드를 통해 찾은 4개의 가장 가까운 픽셀들의 값이며, 가중치인 1-dx, dx 등은 해당 픽셀과 실제 좌표의 거리를 곱하고 있다.

$$val = v_{00}(1 - dx)(1 - dy) + v_{01}dx(1 - dy) + v_{10}(1 - dx)dy + v_{11}dxdy$$

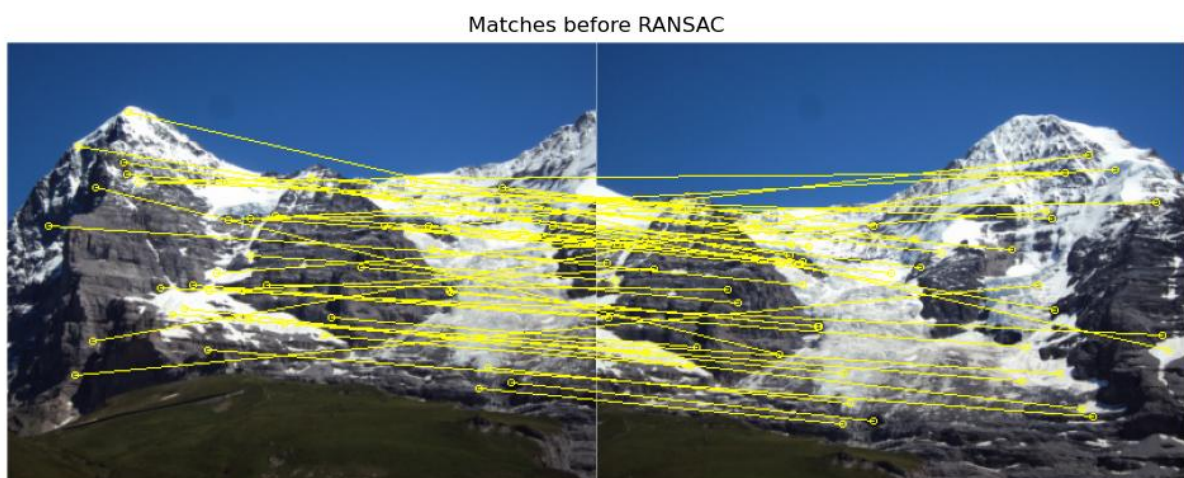
모든 픽셀에 동일한 과정을 거치며 out에는 채널별로 픽셀의 색상 값이 저장되고, C==1인 경우 흑백 이미지를, 그렇지 않은 경우 out 자체로서 최종적인 파노라마 이미지를 반환한다.

3. Results and Analysis

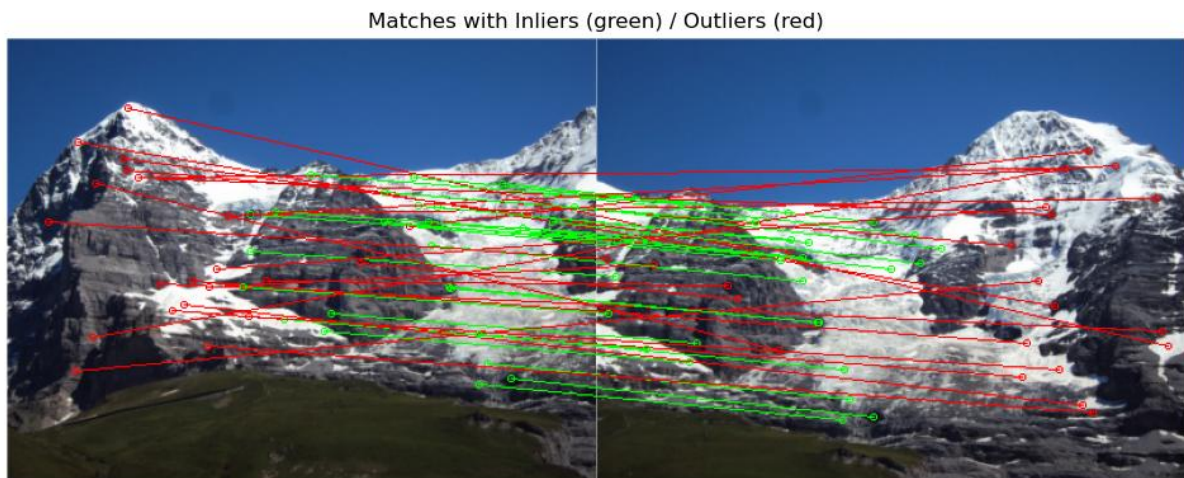
1. SIFT: 세 개의 이미지 각각에 대해 132개, 170개, 205개의 Keypoints가 검출되었다. 아래는 그 중 첫번째 이미지의 Keypoints를 시각화한 결과이다.



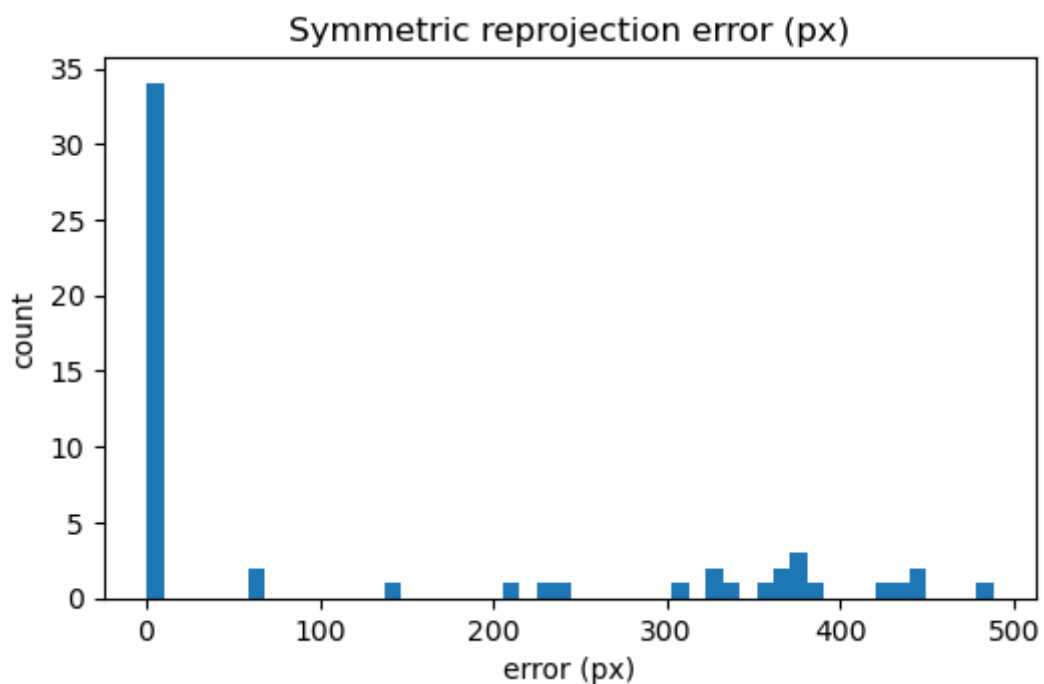
2. Matching: 첫번째와 두번째 이미지 사이에서 56개, 두번째와 세번째 이미지 사이에서 100개의 Keypoints가 매칭되었다. 아래는 그 중 첫번째, 두번째 이미지의 Keypoints 매칭을 시각화한 결과이다.



3. RANSAC + Homography: RANSAC 알고리즘을 적용 후 매칭을 다시 시각화한 결과는 아래와 같다.

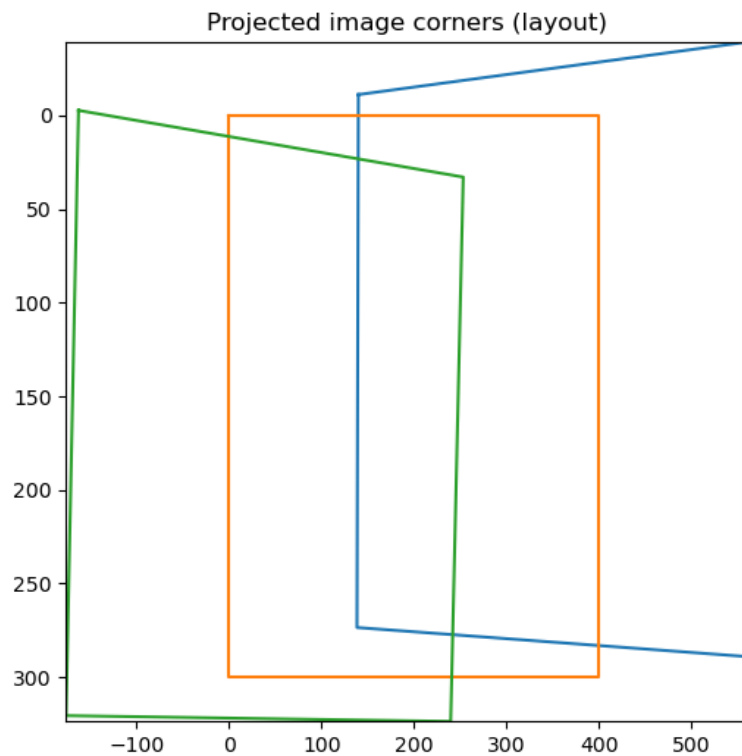


추가로 다음의 히스토그램을 통해 매칭 단계에서 도출된 Keypoint 쌍들의 reprojection error 분포를 확인할 수 있는데,

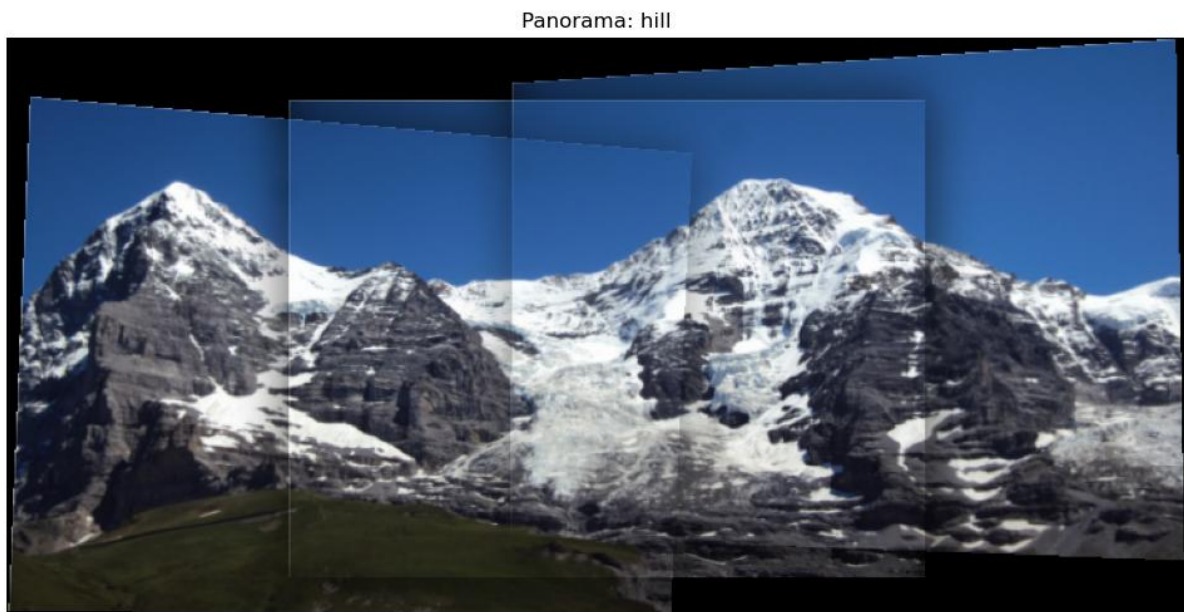


일부가 상당히 높은 수치의 오차를 보유하고 있다. 이를 통해 아웃라이어들을 성공적으로 배제하고 인라이어들을 필터링하는 RANSAC의 중요성을 확인할 수 있다. 구체적인 수치로 첫번째, 두번째 이미지 사이에서는 56개 중 33개, 두번째와 세번째 이미지 사이에서는 100개 중 79개의 Keypoint 쌍이 인라이어로 필터링되었다.

4. Warping, Blending & Panorama: 모든 단계를 거친 후, 각 이미지가 파노라마 캔버스에 projection될 위치는 아래와 같이 도출되었으며,



이 위치에 부합하게 세 이미지가 병합되어 아래의 단일 파노라마 이미지를 성공적으로 출력하였다.



4. Discussion

1. 구현 성과

본 과제를 통해 이미지에서 Keypoint를 검출하고, 이웃한 이미지 간의 비교를 통해 Keypoint를 매칭하였다. 이 때 RANSAC 알고리즘을 통해 인라이어, 아웃라이어를 구분하여 보다 정확한 대응 관계를 구축할 수 있었고, 매칭된 Keypoint 쌍들을 사용해 한 이미지에 대해 다른 이미지를 projection하기 위한 호모그래피 행렬을 계산하였다. 최종적으로 하나의 이미지를 기반으로 파노라마 캔버스를 설정하고 나머지 이미지를 projection하여 단일 파노라마 이미지를 생성하였다.

2. 한계 및 개선

완성도 높은 파노라마를 위해서 이웃한 이미지가 겹치는 경계와 전체 파노라마 이미지의 테두리 등에 대한 개선이 가능할 것으로 보인다. 현재 이미지가 겹치는 부분은 처음의 테두리가 그대로 남아 겹치는 영역에 뚜렷한 이음선이 발생하며, 조명의 불균일성으로 인해 경계 주변에서 밝기 불연속성이 관찰된다. 시각적 완성도를 높이기 위해, 겹치는 영역에 대한 부드러운 전환을 보장하는 페더링 또는 멀티밴드 블렌딩 기법의 도입이 필요하다.

또한 세 개의 이미지를 행렬에 따라 projection하는 것으로 이미지 합성을 바로 완료하였기 때문에 하나의 큰 이미지의 형태로 보기 힘들데, 불규칙한 최종 캔버스 형태를 개선하기 위해, 워핑된 이미지 전체를 포함하는 최소 경계 상자(Minimum Bounding Box)로 이미지를 자르거나, 모자란 영역을 배경값으로 채우는 후처리 과정이 필요하다.

5. Discussion: Parameter Playground

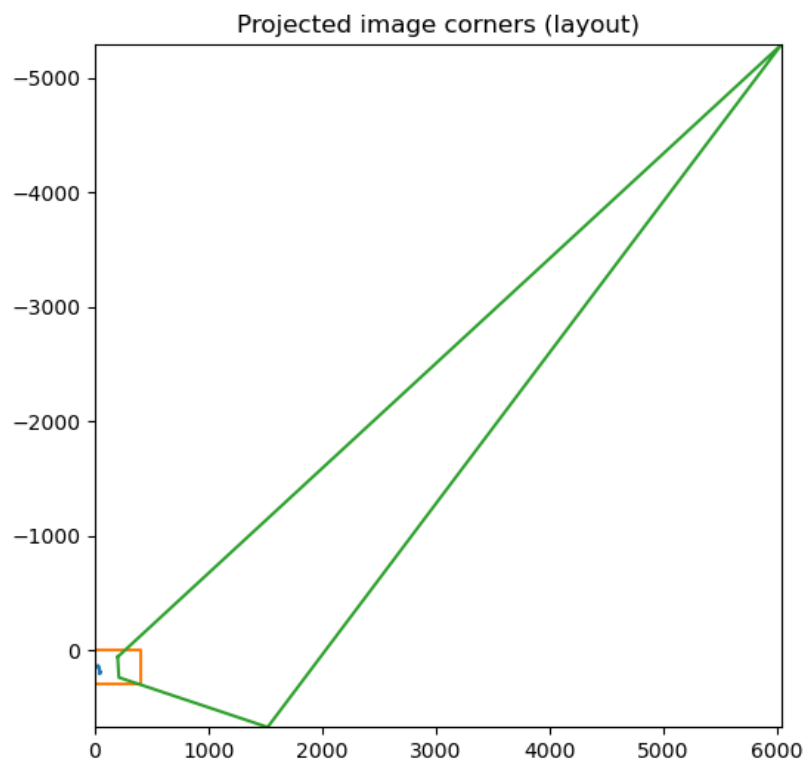
1. SIFT detection

contrast_th (Contrast Threshold)

이 매개변수는 DoG 피라미드에서 극값을 Keypoint의 후보로 넣을지의 여부를 판단하는 대비의 기준이다. contrast_th보다 작은 대비의 점들이 제거되므로, contrast_th 값이 증가할수록 Keypoint는 감소하지만 강력한 특징을 가진 안정적인 점들만 남게 될 것이다. 그 결과 매칭의 수는 감소하지만 그 퀄리티는 증가하여 전체 매칭 수 대비 인라이어의 비율이 증가할 것으로 예상된다. 파노라마 stitching 단계에서 높은 contrast_th는 안정적인 호모그래피 행렬을 도출할 것으로 예상되지만, 값이 너무 커질 경우 충분한 개수의 인라

이어를 확보하지 못할 가능성도 있다. 반대로 낮은 contrast_th는 더 많은 점들을 다루게 되면서 런타임과 RANSAC의 부하를 증가시키고 호모그래피 행렬의 정확도를 감소시킬 가능성이 있다.

매개변수를 0.03에서 0.9로 늘린 결과 132개, 170개, 205개였던 세 이미지의 Keypoint 개수는 각각 10, 14, 12개로 감소하였으며, RANSAC에서 인라이어로 선택되는 비율은 78/100에서 4/5, 33/56에서 8/8로 증가하였다. 하지만 예상과 같이 충분한 개수의 인라이어를 확보하지 못하였기에 정확한 호모그래피 행렬을 도출하지 못하고 부자연스러운 image projection의 결과에 머물렀다.



edge_th (Edge Threshold)

이 매개변수는 $\text{Tr}(H)^2/\text{Det}(H)$ 의 비율을 판단하는 기준으로, $(r+1)^2/r$ 보다 값이 클 경우 reject하므로 edge_th의 값이 증가할수록 엣지 제거의 조건은 완화되고 비교적 코너에 가깝지 않은, 완만한 직선 형태의 점들도 남을 것으로 예상된다. 반대로 값이 감소하면 edge들이 비교적 강력하게 제거되고 꼭짓점에 해당하는 점들이 Keypoint를 구성할 것이다. 낮은 edge_th에 의한 꼭짓점 keypoint들은 3D 시점 변화에 강하기 때문에 안정성이 매우 높을 것이며, 높은 인라이어 비율을 보유할 것이다. 결론적으로 파노라마 정렬에서도 상대적으로 정확한 호모그래피 행렬로 정확한 정렬을 수행할 수 있겠지만, 앞의 contrast_th와 마찬가지로 계산에 활용 가능한 점의 수가 너무 적어질 가능성에 유의해야

한다.

매개변수를 10.0에서 30.0으로 늘린 결과 132개, 170개, 205개였던 세 이미지의 Keypoint 개수는 각각 53, 47, 66개로 감소하였으며, RANSAC에서 인라이어로 선택되는 비율은 78/100에서 21/26, 33/56에서 12/17로 증가하였다.

max_kp (Maximum Keypoints)

이 매개변수는 검출된 Keypoints 중 대비가 높은 순서대로 점들을 고를 때 그 최대 개수의 제한선이다. 높은 max_kp 값은 더 많은 Keypoint를 사용하기 때문에 총 매칭 수는 변수에 선형적으로 증가할 것이며, 그에 따라 더 많은 런타임이 소요될 것이다. 하지만 더 정확한 호모그래피를 계산함으로써 결과적인 파노라마 이미지는 더 높은 정확성을 가질 것으로 추정된다.

매개변수를 1000에서 20으로 줄인 결과 132개, 170개, 205개였던 세 이미지의 Keypoint 개수는 각각 30, 37, 36개로 감소하였으나, RANSAC에서 인라이어로 선택되는 비율은 78/100에서 15/20, 33/56에서 5/13으로 감소하였다. 이는 충분하지 못한 개수의 Keypoint에 의한 결과로 보이며, 실제로 아래와 같이 호모그래피를 정확하게 계산하지 못하고 부자연스러운 결과가 도출되었다.

Panorama: hill



2. Matching

match_method {crosscheck, ratio}

crosscheck와 lowe ratio는 최근접 이웃 탐색 방식에서 조건 만족 여부를 판단하는 두 가지 기준이다. crosscheck는 상호 NN 조건을 만족하는 경우만을 허용하며, lowe ratio는 NN이 두 번째 NN과 충분히 차이나는 경우에는 단방향 매칭도 허용하므로 더 많은 매칭 쌍을 얻을 수 있다. 하지만 건물 창문과 같이 반복 패턴이 많은 경우 비슷한 ratio의 점을 실수로 매칭할 가능성이 있고, 따라서 RANSAC에 의해 결정되는 인라이어 비율은 더 신뢰도가 높은 기준인 crosscheck에서 더 높게 나타날 것이다. 마찬가지로 최종 파노라마의 품질 역시 crosscheck의 쪽이 더 높을 것으로 예상된다.

Stage 2에서 match_l2_crosscheck 함수를 match_love_ratio 함수로 변경해보았으나 차이는 발생하지 않았는데, 이는 애초에 차이를 발생시킬 정도로 Keypoint의 개수가 많지 않으며(다른 매개변수를 조절하며 Keypoint의 개수를 증가시켜보았으니 마찬가지로 차이는 발생하지 않았다), lowe ratio의 단점이 RANSAC에 의해 해결되었기 때문으로 예상된다.

3. RANSAC

ransac_th (Inlier Threshold in pixels)

이 변수는 인라이어, 아웃라이어를 판단하는 오차의 기준값으로 ransac_th보다 작은 오차의 점들이 인라이어로 간주된다. 따라서 낮은 수치는 더 적은 수의, 호모그래피 행렬에 더 정확하게 일치하는 매칭 쌍들을 인라이어로 판단하고, 그 수는 높은 ransac_th에 비해 적을 것이다. 하지만 호모그래피 행렬에 강력하게 일치하는 지점들로 모델을 피팅하기 때문에 모델의 정확도가 높아지며, RANSAC에 성공할 경우 파노라마 역시 높은 정확도로 정렬될 것이다. 하지만 이전 매개변수들과 마찬가지로 과도하게 낮은 값 설정에 의해 인라이어의 수를 확보하지 못해 RANSAC의 실패할 가능성을 유의해야 할 것이다.

하지만 ransac_th 역시 매개변수 값을 3.0에서 1.0으로 줄이는 것이 차이를 발생시키지 않았는데, 이는 이미 반영되어 있는 매칭 쌍들이 높은 정확도를 가지고 있었기 때문으로 예상된다.

iters (Number of Iterations)

이 변수는 RANSAC 알고리즘이 샘플링을 반복하는 횟수를 지정한다. RANSAC은 랜덤으로 샘플을 추출하고 평가하여 최적의 샘플을 갱신하기를 반복하는 알고리즘이므로, 반복

의 횟수는 곧 최종 샘플의 품질에 직결된다. 최적의 호모그래피 행렬은 낮은 확률로 샘플링되기 때문에, 알고리즘을 반복하는 횟수 `iters`가 증가할수록 결과의 재현성은 높아진다. 호모그래피의 `robustness` 역시 `iters`에 비례해 증가하지만, 매 반복마다 행렬 구축 및 오차 계산 등을 수행해야 하므로 계산 비용 역시 선형적으로 증가한다.

매개변수를 4000에서 400000으로 늘린 결과 전체 프로그램이 완료되기까지의 실행 시간은 상당히 늘어났지만, Keypoint의 개수와 이미지의 크기 등이 그다지 큰 상황이 아니었기에 호모그래피의 재현성은 기본적으로 높아 이외의 결과에서 차이는 발생하지 않았다.

4. Homography

DLT Normalization (in `geom/dlt.py`)

기본적으로 `geom/dlt.py`의 `dlt_homography` 함수에서는 입력받은 `pts1`, `pts2`를 `normalize_points` 함수를 사용해 원점 중심, 루트 2 정도의 스케일로 정규화하여 사용한다. 하지만 추가적인 구현으로 `use_normalization` 옵션이 `false`일 경우 `n1`, `n2`에 점의 좌표를 그대로 대입하고 역정규화 행렬이 단위행렬이 되도록 실험해볼 수 있다.

정규화를 사용할 경우 A 행렬의 조건 수가 개선되기 때문에 DLT가 보다 안정적인 호모그래피 행렬을 추정할 수 있지만, 정규화를 사용하지 않을 경우 행렬 성분의 스케일 차이가 커지면서 연산 과정에서의 부동소수점 오류가 비교적 커질 것이다. 이로써 호모그래피 및 reprojection의 오류가 커질 것이다. 파노라마 병합 역시 안정적인 호모그래피 행렬을 도출할 수 있는 정규화를 사용한 알고리즘이 높은 품질의 결과를 도출할 것으로 예상된다.

다만 `ransac_homography` 함수 내의 `dlt_homography` 함수에서 `use_normalization` 옵션을 `True`로 설정한 것과 `False`로 설정한 것에서도 차이가 나지 않았는데, 이는 Keypoint의 개수와 이미지의 크기 등이 그다지 큰 상황이 아니었기에 정규화 여부가 큰 영향을 주지 않았던 것으로 예상된다.

Failure Case Identification

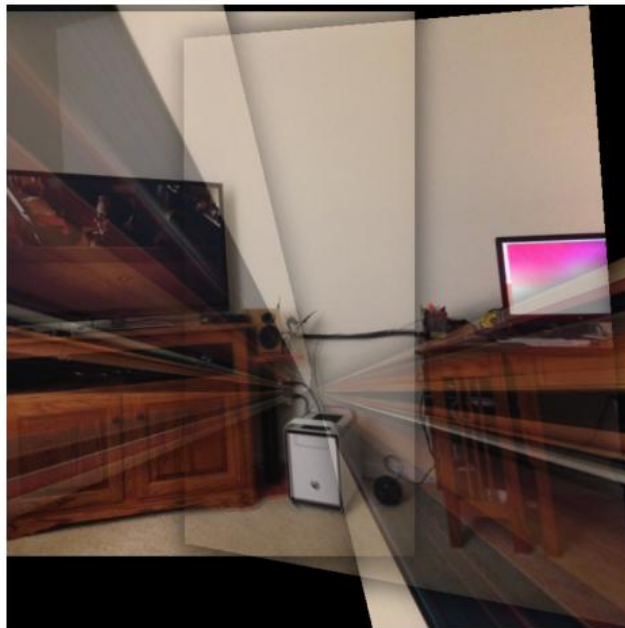
본 과제에는 기본적으로 설정된 hill 이외에도 indoor, keble의 두가지 이미지 데이터가 추가로 있다. keble의 경우 초기의 파라미터 값들로도 아래와 같이 자연스러운 파라노마 이미지를 생성할 수 있었지만,

Panorama: keble

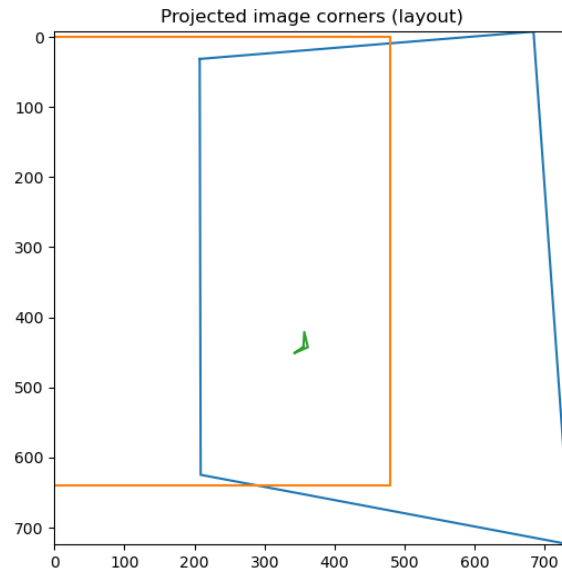


indoor의 경우에는 이미지의 형태가 상당히 찌그러진 어색한 결과물이 도출되었다.

Panorama: indoor

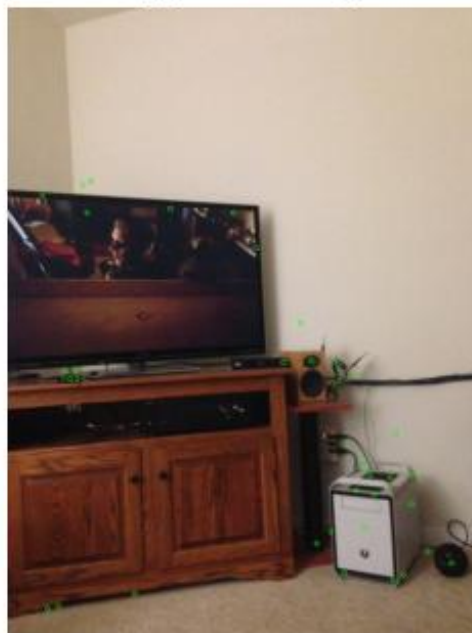


이미지의 배치도를 확인해보면 초록색으로 표현된 한 이미지의 projection이 상당히 이상한 형태로 수행되었음을 확인할 수 있다.



해당 이미지의 호모그래피 추정에서 실패가 발생한 것으로 보이며, 그 첫 번째 원인은 이미지의 넓은 영역이 흰색 벽면을 포함하고 있어 Keypoint가 충분하지 않았기 때문으로 예상하였다. 실제로 각 이미지에서 발견된 Keypoint의 개수는 28, 53, 48개로 상당히 낮았으며, 해당 점들은 아래와 같이 사물 주변의 고대비 영역에 집중되었다.

Keypoints: img 1



이러한 Keypoint 부족은 RANSAC이 A 행렬을 구축할 데이터의 부족과 추정된 호모그래피 행렬의 분산 증가로 수치적 실패를 초래하였다. 추가로, 실패의 근본적인 원인은 단일 호모그래피 모델이 가지고 있는 가정을 위배한 것으로 예상할 수 있다. 단일 호모그래피 모델은 카메라가 고정된 위치에서 순수 회전을 통해 찍은 다수의 사진을 행렬을 통해 projection하는 모델로, 사진들 사이에 시차(Parallax)가 존재할 경우 호모그래피 행렬은

카메라의 회전과 사진 간의 시차를 동시에 처리할 수 없다. 따라서 indoor 이미지 세트는 3장의 사진이 촬영 시에 미세한 이동에 의한 시차가 발생하였고, 이에 따라 모델의 가정을 위배하여 정확한 파노라마 생성에 실패하였다고 판단된다.

하지만 후속 조치로 더 많은 Keypoint를 얻기 위한 조치로 contrast_th를 0.005 줄이고, max_kp를 3000으로 늘인 결과 더 많은 실행 시간이 소요되긴 하였으나 더 많은 Keypoint와 많은 매칭, 그리고 최종적으로 아래와 같이 보다 자연스러운 파노라마 이미지를 도출할 수 있었다.

Panorama: indoor

